# EduGenie: AI-Powered Educational Assistant

## Project Report

### *1. Abstract*

EduGenie is a lightweight web-based educational assistant designed to make study support more accessible. Students can ask questions, request simpler explanations, generate quizzes, build a learning path, and summarize study passages. The application uses FastAPI for HTTP APIs and an HTML, CSS, and JavaScript frontend. It supports an optional cloud AI provider and includes a transparent demo fallback when credentials are not configured.

### *2. Problem Statement*

Students often spend time searching across many sources, struggle with unfamiliar concepts, and lack quick ways to test understanding or organize a learning plan. EduGenie brings several study workflows into one simple interface.

### *3. Objectives*

• Provide concise educational question answering.

• Explain difficult topics in beginner-friendly language.

• Generate multiple-choice practice quizzes with explanations.

• Create time-based learning roadmaps.

• Summarize longer educational passages.

• Keep the application lightweight and easy to run locally.

### *4. Proposed System*

The user interacts with a browser-based interface. JavaScript sends JSON requests to FastAPI endpoints. A service layer optionally calls a configured cloud model; if no model key is available or a request fails, a clearly labeled fallback response is returned. The backend validates input with Pydantic models.

### *5. Main Modules*

1. Ask AI: question-and-answer support.

2. Explain: simplified explanations at beginner, intermediate, or advanced levels.

3. Quiz Generator: topic-based multiple-choice questions with answer checks.

4. Learning Path: weekly milestones, topics, and practice tasks.

5. Summarizer: short or detailed summaries of pasted study text.

### *6. Technology Stack*

• Python 3.10+

• FastAPI and Uvicorn

• Pydantic validation

• Jinja2 templates

• HTML5, CSS3, JavaScript

• Optional OpenAI-compatible cloud model integration

• Pytest and HTTPX for tests

## *7. System Requirements*

Minimum: a computer capable of running Python 3.10+, 2 GB RAM recommended, modern browser, and internet access for cloud AI. The basic demo fallback can run without a cloud key. Apple Silicon systems such as Mac M1 are supported by the Python-based stack, subject to the installed Python package wheels.

## *8. Scenarios*

Scenario 1 — General knowledge: A student asks, “Which is the largest ocean?” EduGenie returns that the Pacific Ocean is the largest.

Scenario 2 — Quiz practice: A student enters “The Pythagoras theorem” and generates multiple-choice questions. They select an answer, check it, and read an explanation.

Scenario 3 — SQL learning path: A learner enters SQL, selects a duration and level, and receives weekly milestones and practice tasks. With cloud AI configured, the plan can be tailored further.

## *9. API Design*

Method • Endpoint • Purpose

GET • / • Render the web interface

GET • /health • Service health check

POST • /api/ask • Answer a question

POST • /api/explain • Explain a concept

POST • /api/quiz • Generate a quiz

POST • /api/learning-path • Create a learning path

POST • /api/summarize • Summarize educational text

## *10. Security and Responsible Use*

• Keep API keys in environment variables; never hard-code or publish them.

• Do not submit private or sensitive information.

• Treat generated responses as study assistance, not guaranteed truth.

• Validate AI outputs and verify important facts with trusted educational sources.

• Consider rate limits, authentication, logging controls, and content filtering before public deployment.

## *11. Testing*

Automated tests cover the home page, health endpoint, a known demo question, and validation of quiz limits. Run tests with pytest.

## *12. Limitations*

• Demo fallback is intentionally limited and is not a real AI model.

• Cloud model quality and availability depend on the configured provider, credentials, network, and quota.

• Generated quiz correctness should be reviewed by a teacher or learner.

• The prototype does not include user accounts, saved progress, or a database.

## *13. Future Enhancements*

• User login and saved learning history.

• PDF and document upload with safe text extraction.

• Progress dashboards and spaced-repetition flashcards.

• Multilingual explanations, including Tamil.

• Teacher dashboards and curated course content.

• More robust model-output schemas, rate limits, and deployment monitoring.

## *14. Conclusion*

EduGenie combines five common study workflows in a small web application. Its modular backend and simple interface make it suitable as a student project prototype and a foundation for further development.